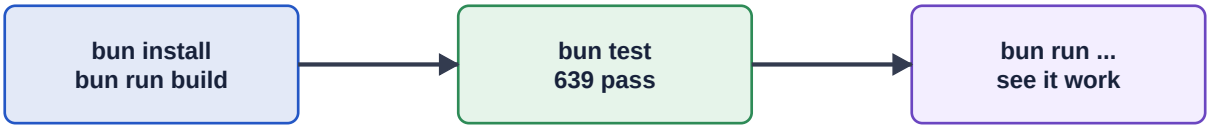
HANDS-ON LAB GUIDE FOR THE petri-flow REPOSITORY

PetriFlow, run it yourself

Every command for Git Bash on a Windows laptop, in order. For each one: what it does, which functions execute, a diagram of the flow, and the exact output you should see. Copy, paste, run, understand.

Every command here was executed on 9 October 2026 on Windows 11 (Bun 1.3.14, Node 24, Git 2.49).



WHAT IS INSIDE

Part 1 Set up the laptop, clone, install, build, typecheck (steps 1.1 to 1.8)
Part 2 Run every test suite, package by package, and read what each one proves (2.1 to 2.10)
Part 3 Run the six workflows and the analysis CLI (3.1 to 3.6)
Part 4 Drive the safety gate by hand with a 40-line script, no API key (4.1 to 4.2)
Part 5 The viewer, the comparison reports, and the optional LLM agents (5.1 to 5.5)
Part 6 Break a proof on purpose and watch the tests catch it (6.1 to 6.2)
Part 7 Troubleshooting, cleanup, and a one-page command cheat sheet

Companion to 'PetriFlow, explained from zero', which covers the concepts and architecture.
This guide is about doing. Where a concept is needed, it points at the chapter in the first guide.

Contents

Contents 2

How to use this guide 3

Part 1. Set up, clone, install, build 4

Part 2. Run every test suite and read what it proves 8

Part 3. Run the workflows and the analysis CLI 14

Part 4. Drive the safety gate by hand (no API key) 18

Part 5. The viewer, the comparison reports, and the agents 21

Part 6. Break a proof on purpose 24

Part 7. Troubleshooting, cleanup, cheat sheet 26

7.1 If something goes wrong 26

7.2 Leaving the repo clean 26

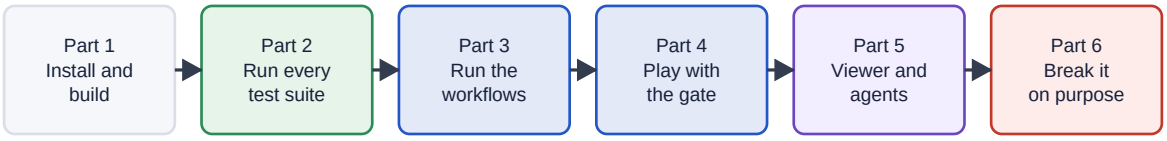
7.3 One-page cheat sheet, in order 27

7.4 Where each command's code lives 27

How to use this guide

Work through the parts in order. Each step has the same three pieces:

- A **dark command box**. Copy it exactly into Git Bash. All commands assume you are in the repository root folder unless the step says otherwise.
- A **'What is executing' box**. It names the files and functions that run when you press Enter, so the command is never a black box.
- A **green 'Expected output' box**. It shows what this guide saw when the command ran. Small differences in timing, versions and paths are normal.



Each part is a set of numbered steps. Every step has: the command, what runs inside, what you should see.

The route through the guide. Parts 1 and 2 are required. The rest can be done in any order.

> **NOTE Git Bash on Windows: the three things that will bite you**
>
> 1) Paths. Git Bash shows `/c/Users/you` but programs started from it (Bun, Node) are Windows programs that want `C:\Users\you`. Some repo code assumes a Unix path and breaks. Part 2 gives one-line fixes. 2) `/tmp`. Inside Git Bash it exists. For Bun it means `C:\tmp`, which normally does not exist. Step 1.6 creates it. 3) Corporate proxy. If `bun install` or the GitHub lookup hangs, set the proxy variables shown in step 1.1.

Time needed: about 25 minutes for Parts 1 and 2, then 5 to 10 minutes per later part. Disk: around 600 MB for node_modules. No API key is needed for anything except Part 5.5.

Part 1. Set up, clone, install, build

Step 1.1 Check the tools on the laptop

```
git --version
node --version
bun --version
```

Expected output (your numbers may be newer)

```
git version 2.49.0.windows.1
v24.11.0
1.3.14
```

Bun is the only unusual requirement. It is the runtime, the package manager and the test runner for this repo. If the last line says *bun: command not found*, install it with one of these, then **open a new Git Bash window**:

```
npm install -g bun
# or, if npm is not allowed:
powershell -c "irm bun.sh/install.ps1 | iex"
```

If your company uses a proxy, set it before any install. Ask IT for the host and port.

```
export HTTPS_PROXY=http://proxy.yourcompany.com:8080
export HTTP_PROXY=http://proxy.yourcompany.com:8080
git config --global http.proxy http://proxy.yourcompany.com:8080
```

WHAT IS EXECUTING

Nothing from the repo yet. These just check or configure your shell. Bun reads HTTPS_PROXY for downloads. Git reads its own http.proxy setting.

Step 1.2 Clone and enter the repo

```
cd ~
git clone https://github.com/joshuaisaact/petri-flow.git
cd petri-flow
ls
```

Expected output

```
BENCHMARK.md  CHANGELOG.md  CLAUDE.md  LICENSE  README.md  apps  bun.lock  comparisons
examples  package.json  packages  site  tsconfig.json  turbo.json  workflows
```

If you already cloned it somewhere else, just `cd` into that folder. Every later command starts from here. If a step changes folder, the next step says how to come back.

Step 1.3 Install dependencies

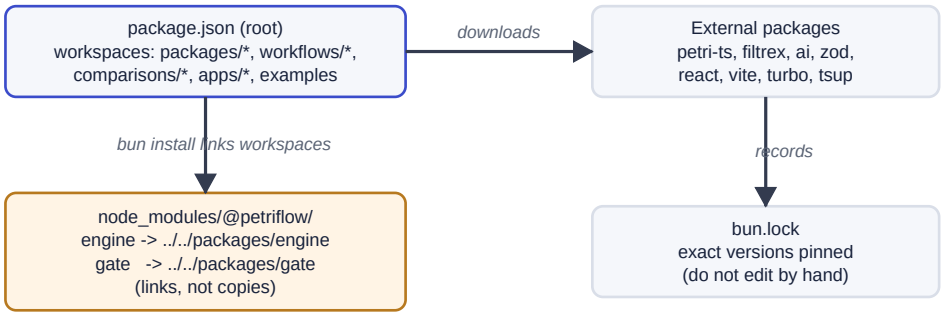
```
bun install
```

Expected output

```
bun install v1.3.14
...
+ turbo@2.x, tsup@8.x, typescript@5.x, ai@6.x ...
(~600 packages installed, 20 to 90 seconds)
```

WHAT IS EXECUTING

Bun reads the root package.json, finds the *workspaces* list (packages/*, workflows/*, comparisons/*, apps/*, examples), downloads external packages into node_modules, and creates links so that `@petriflow/gate` points at `packages/gate`. bun.lock pins exact versions. No repo code runs.



After bun install, 'import from @petriflow/gate' resolves to packages/gate/dist (the built output), not src.

What bun install produces. The @petriflow/* entries are links into the repo, not downloads.

Step 1.4 Build the packages (required before tests)

```
bun run build
```

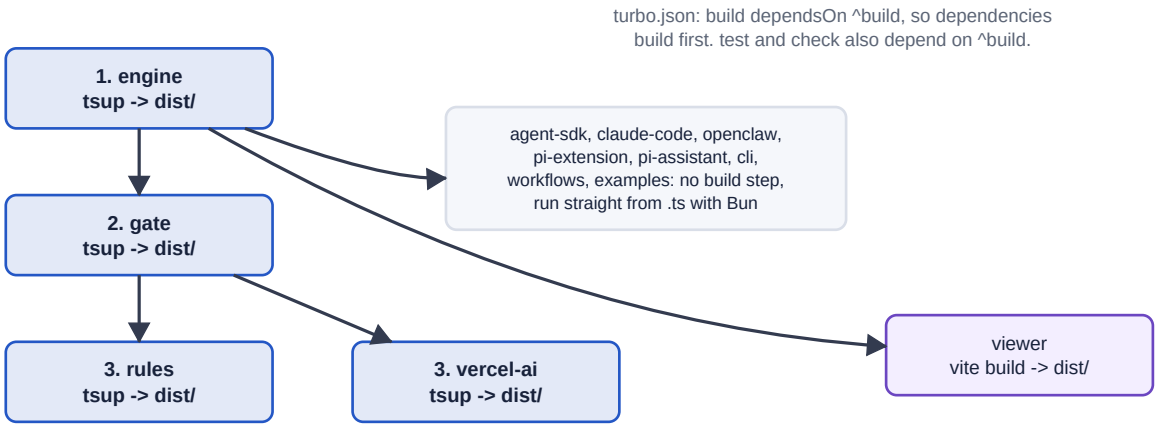
Expected output

```
@petriflow/engine:build: ESM dist/index.js ...
@petriflow/gate:build: ...
@petriflow/rules:build: ...
@petriflow/vercel-ai:build: ...
@petriflow/viewer:build: vite v6 building for production...

 Tasks:    8 successful, 8 total
  Time:    31.795s
```

WHAT IS EXECUTING

`turbo run build` walks the dependency graph from turbo.json. For engine, gate, rules and vercel-ai it runs **tsup**, which compiles src/*.ts into dist/ (ESM, CommonJS and .d.ts). For the viewer it runs **vite build**. The other packages have no build step and run straight from TypeScript. This step matters because every package imports `@petriflow/engine` through its package.json *exports*, which point at dist/. No dist, no imports, no tests.



Build order. Turbo builds engine first because everything depends on it.

LOOPHOLE The README skips this step

The quick start says 'bun install' then 'bun test'. On a fresh clone that fails every suite with *Cannot find module '@petriflow/gate'*. Always build first.

Step 1.5 Typecheck everything

```
bun run check
```

Expected output

```
@petriflow/engine:check: $ tsc --noEmit
...
 Tasks:    15 successful, 15 total
  Time:    9.039s
```

WHAT IS EXECUTING

`tsc --noEmit` in every workspace. It compiles nothing to disk; it only reports type errors. Strict mode is on, including noUncheckedIndexedAccess, so `arr[0]` is typed as possibly undefined, which is why the code is full of `arr[0]!`.

Step 1.6 Windows preparation: create C:\tmp

```
mkdir -p /c/tmp
ls -d /c/tmp
```

Expected output

```
/c/tmp
```

WHAT IS EXECUTING

Nothing from the repo. The claude-code adapter (packages/claude-code/src/state.ts) writes its state file to the literal path `/tmp/petriflow-claude-code-<session>.json`. Windows resolves a path starting with a slash against the current drive, so `/tmp/x` becomes `C:\tmp\x`. Creating the folder makes 8 tests pass that otherwise fail with ENOENT. This was verified: 10 pass / 8 fail before, 18 pass / 0 fail after.

Step 1.7 First full test run (expect some Windows failures)

```
bun test 2>&1 | tail -5
```

Expected on Windows after step 1.6

```
 588 pass
 9 fail
 4 errors
 2389 expect() calls
Ran 597 tests across 39 files. [19.39s]
```

Nine failures and four errors remain, and all of them are Windows path problems in test helpers, not in the gate logic. Part 2 explains each and gives the two one-line patches that take the suite to **639 pass, 0 fail**. On macOS or Linux you would see 639 pass right here.



'bun test packages/gate' skips turbo and runs that folder directly. Faster, and the output is not interleaved.
The final lines always read: N pass / N fail / Ran N tests across N files.

What 'bun test' does at the root versus in one folder.

Step 1.8 Optional: see the whole dependency graph Turbo uses

```
bunx turbo run build --dry-run=json 2>/dev/null | grep -E \
  '"taskId"|"dependencies"' | head -30
```

WHAT IS EXECUTING

Turbo prints its plan without running anything. Each taskId is a workspace plus task. The dependencies arrays are the ^build edges. Useful when you wonder why a package rebuilt.

Part 2. Run every test suite and read what it proves

Run one package at a time. Each command below runs in well under a second except the workflows (real timers) and claude-code (spawns processes). Read the describe names in the output: they are the specification of the package in plain words.

Step 2.1 Engine: the workflow runtime (125 tests, 12 files)

```
bun test packages/engine
```

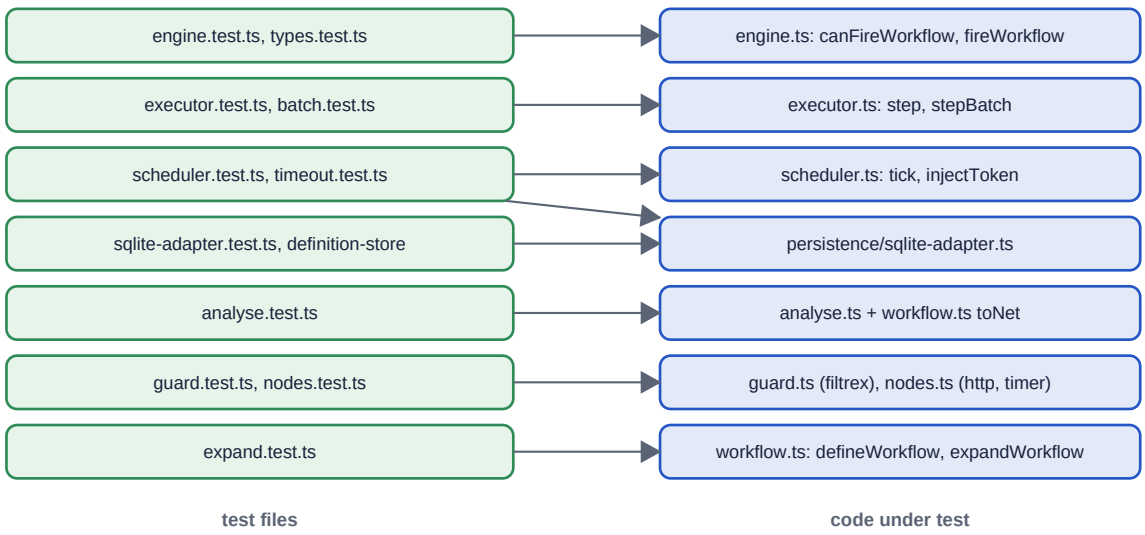
Expected output

```
bun test v1.3.14

packages\engine\src\engine.test.ts:
(pass) canFireWorkflow > returns true when structurally enabled and guard passes
...
 125 pass
 0 fail
Ran 125 tests across 12 files. [752.00ms]
```

WHAT IS EXECUTING

Each test file targets one module. engine.test.ts calls `canFireWorkflow` and `fireWorkflow` directly with tiny nets. scheduler.test.ts builds a 4-place pipeline, creates an in-memory SQLite database, and drives `Scheduler.tick()` step by step, asserting the onFire order and the final status. timeout.test.ts schedules a timeout and advances the clock. nodes.test.ts exercises the HTTP node, including the private-host (SSRF) block. analyse.test.ts checks reachable state counts and invariants.



Engine test files and the modules they exercise.

Try one file alone to see the full list of assertions:

```
bun test packages/engine/src/scheduler.test.ts
```

Step 2.2 Gate: the safety core (107 tests, 3 files)

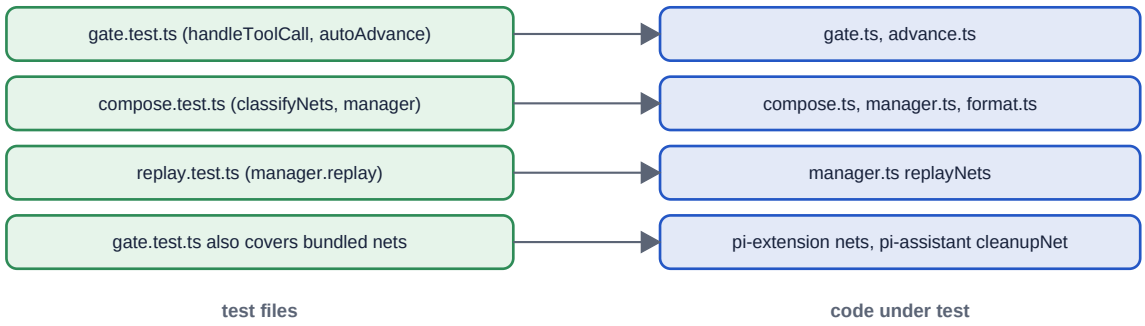
```
bun test packages/gate
```

Expected on Windows before the patch in step 2.9 (107 pass after)

```
 104 pass
 3 fail
Ran 107 tests across 3 files.
```

WHAT IS EXECUTING

gate.test.ts covers `handleToolCall` for one net: free tools, blocked tools, manual approval yes/no/no-UI, and the TOCTOU re-check. It also covers `autoAdvance`, including the rule that two competing structural transitions both stay put. compose.test.ts covers `classifyNets` (free/abstain/gated/blocked), the GateManager in array and registry mode, deferred transitions, meta rollback, shadow mode and onDecision. replay.test.ts covers `manager.replay`. The 3 Windows failures are in 'path extraction' and 'nukeNet' tests that import the pi-extension nuke net, whose path normaliser turns 'src/' into 'src\'.



Gate test files. The 4-phase protocol from the first guide, chapter 6, is tested here.

Step 2.3 Rules: the DSL compiler (67 tests)

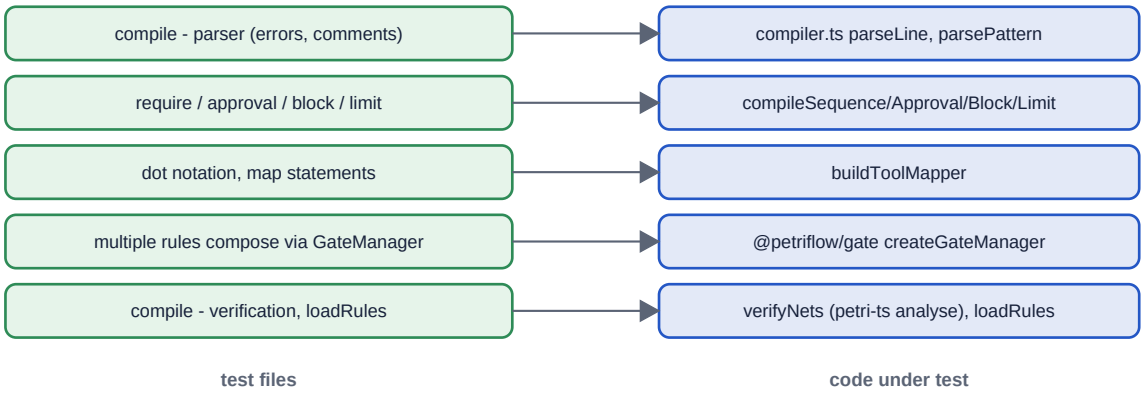
```
bun test packages/rules
```

Expected output

```
 67 pass
 0 fail
Ran 67 tests across 1 file. [75.00ms]
```

WHAT IS EXECUTING

One file, compiler.test.ts. The parser section feeds bad lines and asserts the exact error text with the line number. Each rule type gets a section that compiles a rule, builds a GateManager and plays a scenario: 'require A before B' blocks B, allows A (deferred), resolves the result, then allows B. The limit section counts down a budget. The verification section checks the reachable state numbers: 3 for a sequence rule, 2 for approval, 2 for block, N+2 for a session limit.



Rules tests. Every DSL template from the first guide, chapter 3, has a scenario here.

Step 2.4 Vercel AI adapter (64 tests)

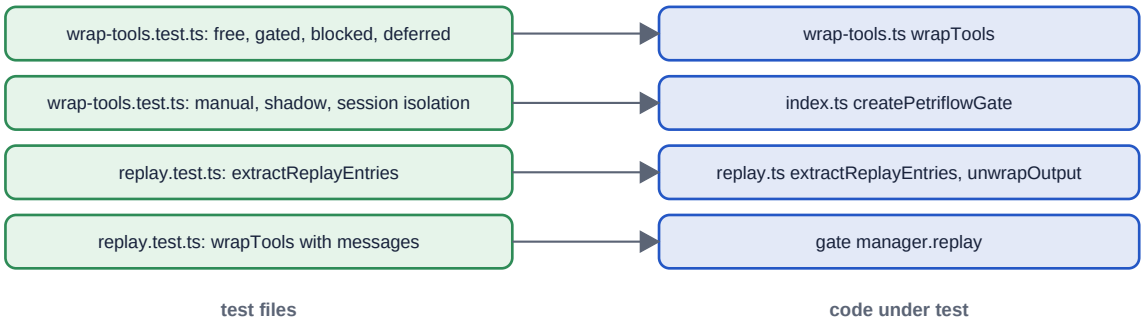
```
bun test packages/vercel-ai
```

Expected output

```
 64 pass
 0 fail
Ran 64 tests across 2 files. [55.00ms]
```

WHAT IS EXECUTING

wrap-tools.test.ts creates fake tools with an execute function, wraps them with `createPetriflowGate().wrapTools()` and calls execute directly, no LLM involved. It checks that blocked tools throw `ToolCallBlockedError`, that deferred nets only advance when isToolResultError says the result is fine, and that two wrapTools() sessions do not share state. replay.test.ts builds fake message arrays with tool-call and tool-result parts and checks that `extractReplayEntries` pairs them by toolCallId and classifies SDK error outputs.



Vercel AI adapter tests.

Step 2.5 CLI (4 tests)

```
bun test packages/cli
```

Expected output

```
 4 pass
 0 fail
Ran 4 tests across 1 file.
```

WHAT IS EXECUTING

test/analyse.test.ts defines a 3-place pipeline inline and calls `analyse()` from the engine, checking reachable states, terminal states and two invariants (one that holds, one that does not). It does not spawn the CLI binary; Part 3 does that by hand.

Step 2.6 The other adapters: agent-sdk, pi-extension, openclaw, pi-assistant

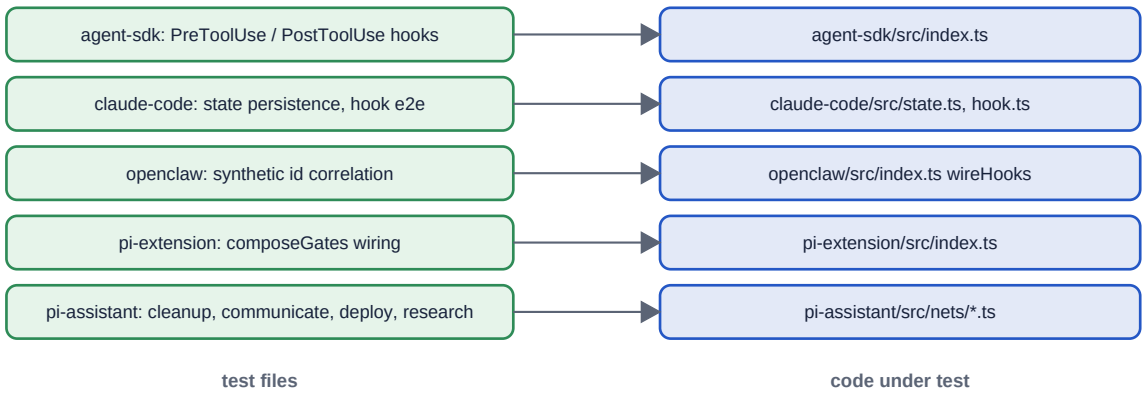
```
bun test packages/agent-sdk
bun test packages/pi-extension
bun test packages/openclaw
bun test packages/pi-assistant
```

Expected summary lines

```
agent-sdk:     21 pass, 0 fail
pi-extension:  19 pass, 0 fail
openclaw:      20 pass, 0 fail
pi-assistant:  57 pass, 2 fail   (59 pass after the patch in step 2.9)
```

WHAT IS EXECUTING

agent-sdk tests call the returned PreToolUse and PostToolUse callbacks with the same JSON shape the Claude Agent SDK sends, and check the deny decision text. pi-extension and openclaw tests build a fake host object with `on()` and `registerCommand()`, install the adapter, then emit events. The openclaw suite specifically tests the synthetic id FIFO: two calls of the same tool, results in order. pi-assistant tests the four hand-written nets; the two Windows failures are the path extractor returning 'src\' instead of 'src'.



Adapter tests. Each one fakes its host framework so no real framework is needed.

Step 2.7 Claude Code adapter (18 tests, needs step 1.6)

```
bun test packages/claude-code
```

Expected after mkdir -p /c/tmp (otherwise 10 pass, 8 fail with ENOENT)

```
 18 pass
 0 fail
Ran 18 tests across 1 file.
```

WHAT IS EXECUTING

hook.test.ts calls `saveState` / `restoreState` / `clearState` from state.ts with a test session id, and simulates PreToolUse then PostToolUse across 'process boundaries' by creating a brand-new GateManager and restoring into it, exactly as hook.ts does for each event. The end-to-end test proves a deferred transition survives being written to disk and read back.

Step 2.8 Workflows and comparisons (40 + 49 tests)

```
bun test workflows
bun test comparisons
```

Expected summary lines

```
workflows:    40 pass, 0 fail, Ran 40 tests across 6 files. [17.76s]
comparisons:  49 pass, 0 fail, Ran 49 tests across 2 files.
```

WHAT IS EXECUTING

Each workflow test does two things: **run** it with a real Scheduler and in-memory SQLite and assert the fired order and final context, and **analyse** it and assert properties over all reachable states. The 17 seconds are real: the coffee executors sleep 2, 1 and 3 seconds to imitate work. github-lookup has a 'live' test that calls api.github.com; behind a proxy without the variables from step 1.1 that one test fails with a fetch error. agent-benchmark.test.ts is the headline: four properties over 196 states (first guide, chapter 9.5). comparisons/n8n counts LLM calls (2 batched vs 11 ReAct). comparisons/openclaw-safety proves the four OpenClaw scenarios.

Step 2.9 Make the whole suite green on Windows (two safe, reversible patches)

Both failures are in code that builds file paths. The fixes are one line each and do not change behaviour on Linux or macOS. Apply them, run the suite, then revert with git if you want the tree clean.

```
sed -i 's|from "node:path";|from "node:path/posix";|' packages/pi-assistant/src/nets/cleanup.ts \
  packages/pi-extension/src/nets/nuke.ts
FIX='s|new URL(\(.*\), import.meta.url).pathname|Bun.fileURLToPath(new URL(\1, import.meta.url))|'
sed -i "$FIX" examples/0*/agent.test.ts
git diff --stat
```

Expected output

```
 examples/01-file-management/agent.test.ts         | 2 +-
 examples/02-deployment/agent.test.ts              | 2 +-
 examples/03-discord-bot/agent.test.ts             | 2 +-
 examples/04-devops-assistant/agent.test.ts        | 2 +-
 packages/pi-assistant/src/nets/cleanup.ts         | 2 +-
 packages/pi-extension/src/nets/nuke.ts            | 2 +-
 6 files changed, 6 insertions(+), 6 deletions(-)
```

WHAT IS EXECUTING

Patch 1 makes the path normaliser use forward slashes everywhere (`node:path/posix`), which is what the regexes in those nets expect. Patch 2 replaces `new URL(...).pathname`, which yields '/C:/petri-flow/...' on Windows, with `Bun.fileURLToPath`, which yields 'C:\petri-flow\...'. Both were verified on this machine.

```
bun test 2>&1 | tail -4
```

Expected after both patches and step 1.6

```
 639 pass
 0 fail
 2568 expect() calls
Ran 639 tests across 39 files. [19.20s]
```

The count went from 597 to 639 because the four example suites (46 tests) could not even load before. To undo the patches later:

```
git checkout -- examples packages/pi-assistant packages/pi-extension
git status --short
```

Step 2.10 Examples (46 tests, after patch 2)

```
bun test examples
```

Expected output

```
 46 pass
 0 fail
Ran 46 tests across 4 files. [66.00ms]
```

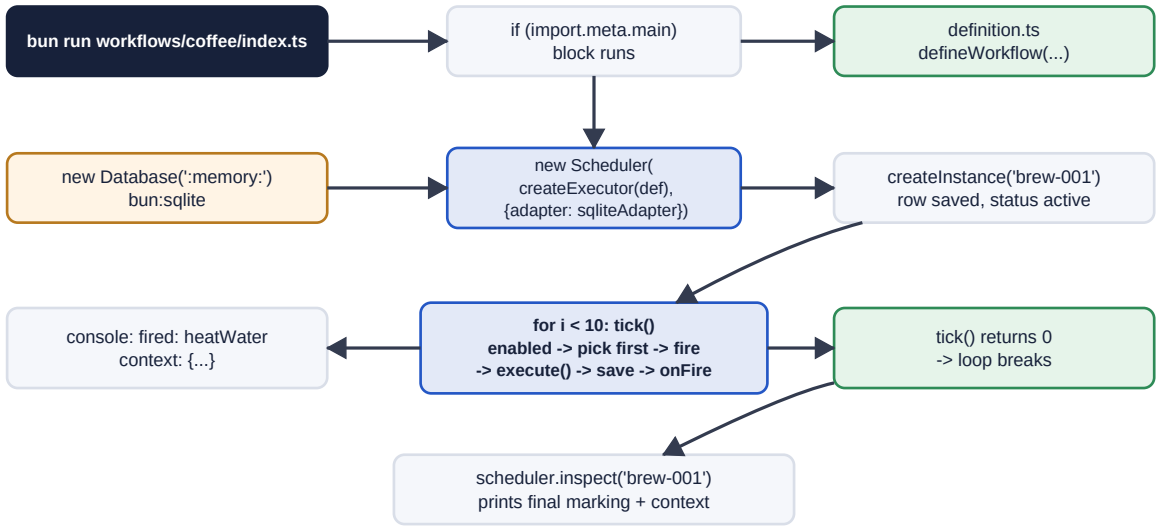
WHAT IS EXECUTING

Each example folder has agent.test.ts that loads the real .rules file next to it, builds the gate with mock tools, and plays the scenario from its README without any LLM: 01 blocks rm and requires backup before delete; 02 enforces lint, test, deploy with approval and a limit of 2; 03 uses dot-notation Discord actions; 04 composes 10 rules across 13 tools and checks they do not interfere. These are the best tests to read if you want to see the DSL used realistically.

Command	Tests	Seconds	Needs
bun test packages/engine	125	under 1	build
bun test packages/gate	107	under 1	build, patch 1 for 3 tests
bun test packages/rules	67	under 1	build
bun test packages/vercel-ai	64	under 1	build
bun test packages/cli	4	under 1	build
bun test packages/agent-sdk	21	under 1	build
bun test packages/pi-extension	19	under 1	build
bun test packages/openclaw	20	under 1	build
bun test packages/pi-assistant	59	under 1	build, patch 1 for 2 tests
bun test packages/claude-code	18	under 1	build, /c/tmp
bun test workflows	40	about 18	build, internet for 1 test
bun test comparisons	49	under 1	build
bun test examples	46	under 1	build, patch 2
bun test (everything)	639	about 20	all of the above

Part 3. Run the workflows and the analysis CLI

Tests prove things. Running the workflows lets you watch the scheduler move tokens. Every workflow index.ts has the same shape: export the definition, and if the file is run directly, build a Scheduler and tick it.



What happens when you run a workflow file directly. Same shape for all six.

Step 3.1 Coffee: two parallel branches and a join

```
bun run workflows/coffee/index.ts
```

Expected output (takes about 6 seconds because executors sleep)

```
[brew-001] fired: heatWater
  context: { waterTemp: 96, grindSize: "none", brewed: false }
[brew-001] fired: grindBeans
  context: { waterTemp: 96, grindSize: "medium", brewed: false }
[brew-001] fired: pourOver
  context: { waterTemp: 96, grindSize: "medium", brewed: true }
[brew-001] v completed

Final state: { marking: { ..., coffeeReady: 1 }, status: "completed" }
```

WHAT IS EXECUTING

Tick 1: enabled = [heatWater, grindBeans]. No decision provider is set, so `step()` picks the first, heatWater; its execute sleeps 2 s and returns {waterTemp: 96}. Tick 2: grindBeans. Tick 3: pourOver is now enabled (three input tokens) and its guard 'waterTemp >= 90' passes. Tick 4: nothing enabled, no timers, status becomes completed, the loop breaks. Each tick saved a row to SQLite and appended to transition_history.

IN SIMPLE WORDS Why did heatWater go first?

Because it is listed first in definition.ts and no decision provider is configured. The analyser treats both orders as possible (5 reachable states). The runtime picked one. That gap between 'all possible' and 'what happened' is the whole point of analysing before running.

Step 3.2 Order checkout: a resource pool

```
bun run workflows/order-checkout/index.ts
```

Expected output

```
[order-001] fired: reserve_stock
  marking: { order_placed: 0, inventory: 2, reserved: 1, payment: 1, shipped: 0, out_of_stoc
    k: 0 }
[order-001] fired: process_payment
  marking: { order_placed: 0, inventory: 2, reserved: 0, payment: 0, shipped: 1, out_of_stoc
    k: 0 }
[order-001] v completed
```

WHAT IS EXECUTING

inventory starts with 3 tokens. reserve_stock consumes one order token and one inventory token and produces reserved and payment. process_payment consumes both and produces shipped. out_of_stock has the guard 'marking.inventory == 0', which is false here, so it never fires. Watch the invariant: inventory + reserved + shipped is 3 in every line.

Step 3.3 Simple agent and agent benchmark: loops with a budget

```
bun run workflows/simple-agent/index.ts
bun run workflows/agent-benchmark/index.ts 2>&1 | grep -E 'fired|completed'
```

Expected output (benchmark takes about 10 seconds)

```
[agent-001] fired: plan
[agent-001] fired: dispatchTool
[agent-001] fired: completeTool
[agent-001] fired: generate
[agent-001] v completed

[agent-001] fired: plan
[agent-001] fired: distribute
[agent-001] fired: dispatchSearch
[agent-001] fired: completeSearch
[agent-001] fired: dispatchDB
[agent-001] fired: completeDB
[agent-001] fired: dispatchCode
[agent-001] fired: requestApproval
[agent-001] fired: approveCode
[agent-001] fired: executeCode
[agent-001] fired: joinResults
[agent-001] fired: generate
[agent-001] v completed
```

WHAT IS EXECUTING

In simple-agent, resultsReady enables both *generate* and *iterate*. Without a decision provider the executor takes the first, generate, so you see zero iterations and the budget stays 2. In agent-benchmark the plan executor sets useSearch/useDB/useCode to true, so the guards on the dispatch transitions pass and the skip transitions do not. approveCode is a 'manual' transition, but the plain executor does not know about manual; it just fires it. Human gating is enforced by the gate package, not by the engine (first guide, chapter 10).

To see the LLM actually choose between generate and iterate you need an API key: see step 5.5. The file is workflows/simple-agent/run-llm.ts.

Step 3.4 GitHub lookup: real HTTP calls as transitions

```
bun run workflows/github-lookup/index.ts
```

Expected output

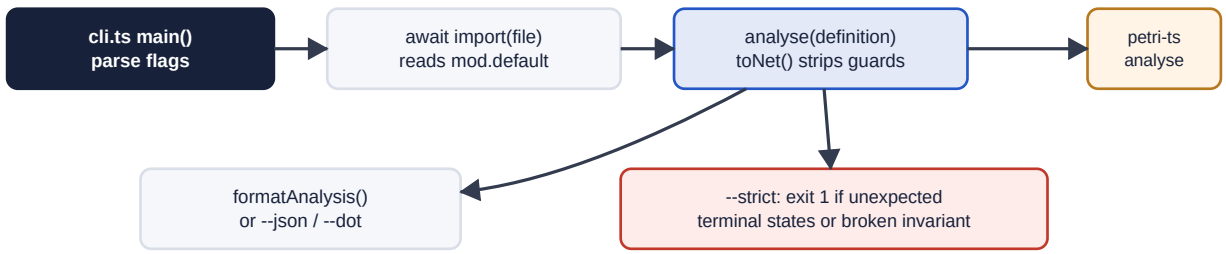
```
[lookup-001] fired: fetchUser
  status: 200 (ok: true)
  user: octocat - The Octocat
  public repos: 8

[lookup-001] fired: fetchRepos
  status: 200 (ok: true)
  top 5 repos:
    - Spoon-Knife *14088 (HTML)
    ...
[lookup-001] v completed - looked up github.com/octocat
```

WHAT IS EXECUTING

Both transitions have type 'http' and a config with a url, so `defineWorkflow` gave them an executor from the node registry (nodes.ts httpNode). httpNode checks the host is not private, fetches with a 30 s timeout and redirect: error, and stores {status, ok, data} under the transition name in context. Behind a corporate proxy without step 1.1 variables you get 'fetch failed'. That is expected and not a repo bug.

Step 3.5 The analysis CLI: prove properties without running



packages/cli/src/cli.ts. The workflow file is imported, not executed; import.meta.main is false so nothing ticks.

```
bun packages/cli/src/cli.ts analyse workflows/coffee/index.ts
bun packages/cli/src/cli.ts analyse workflows/order-checkout/index.ts \
  --strict; echo "exit code: $?"
```

Expected output

```
Workflow: coffee

  Reachable states: 5
  Terminal states:  1 (all valid)
    v coffeeReady

Workflow: order-checkout

  Reachable states: 4
  Terminal states:  2 (all valid)
    v inventory(3), out_of_stock
    v inventory(2), shipped

  Invariants:
    1*inventory + 1*reserved + 1*shipped = const -> holds
exit code: 0
```

WHAT IS EXECUTING

`analyse()` in packages/engine/src/analyse.ts calls `toNet()` to drop guards and executors, then petri-ts enumerates markings breadth-first. Terminal states are split into valid (a terminal place has a token) and unexpected. With --strict the process exits 1 if any unexpected terminal state exists or an invariant fails, which is how you would wire it into CI. Note that the out_of_stock terminal state is reachable in the analysis even though the runtime guard made it impossible in step 3.2. Analysis ignores guards.

```
bun packages/cli/src/cli.ts analyse workflows/agent-benchmark/index.ts --json \
  | head -5
bun packages/cli/src/cli.ts analyse workflows/coffee/index.ts --dot | sed -n \
  '/--- DOT ---/,$p'
```

Expected output

```
{
  "reachableStateCount": 196,
  "terminalStates": [
    {
      "userQuery": 0,

--- DOT ---
digraph {
  rankdir=LR;
  "waterCold" [shape=circle label="waterCold\n1"];
  ...
  "t2" [shape=box style=filled fillcolor=lightgrey label="pourOver"];
  "waterHot" -> "t2";
  ...
}
```

Paste the DOT text into any Graphviz viewer (for example the online editor at dreampuf.github.io/GraphvizOnline) to get a picture of the net. The CLI help is bun `packages/cli/src/cli.ts --help`.

Step 3.6 Analyse all six workflows in one loop

```
for w in coffee github-lookup contract-approval order-checkout simple-agent \
  agent-benchmark; do
  bun packages/cli/src/cli.ts analyse workflows/$w/index.ts --strict > \
  /dev/null && echo "$w: OK" || echo "$w: FAILED"
done
```

Expected output

```
coffee: OK
github-lookup: OK
contract-approval: OK
order-checkout: OK
simple-agent: OK
agent-benchmark: OK
```

Part 4. Drive the safety gate by hand (no API key)

This is the most useful lab in the guide. A 40-line script compiles three rules, builds a GateManager, and calls it the way an adapter would. You watch every verdict and every marking change.

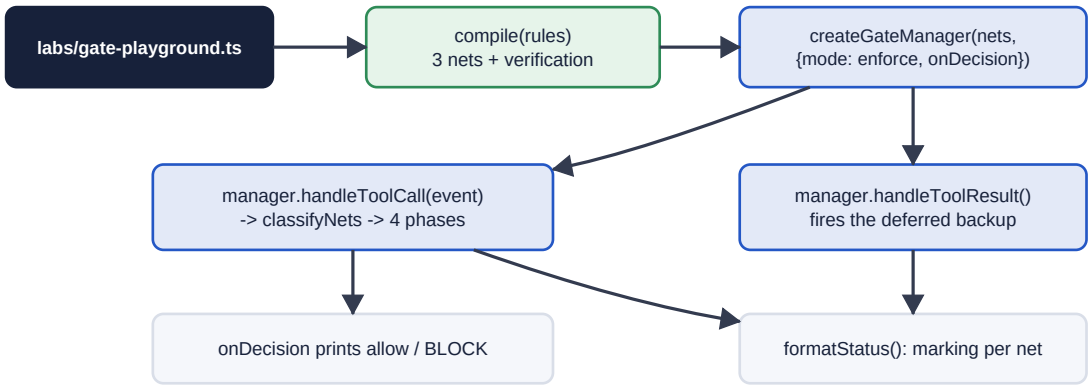
Step 4.1 Create the script

Create a folder `labs` in the repo root and save this as `labs/gate-playground.ts`. The repo root resolves @petriflow/* because of the workspace links from step 1.3.

```
mkdir -p labs
cat > labs/gate-playground.ts <<'EOF'
// Lab: play with the gate without any API key.
// Run from the repo root:  bun run labs/gate-playground.ts
import { compile } from "@petriflow/rules";
import { createGateManager } from "@petriflow/gate";
// 1. Compile three rules. Each becomes one small Petri net.
const { nets, verification } = compile(`
  require backup before delete
  block rm
  limit push to 2 per session
`);
console.log("Compiled nets:", nets.map((n) => n.name));
console.log("Verification (reachable states per net):", verification);
// 2. Build a manager. "enforce" means blocked calls really get blocked.
const manager = createGateManager(nets, {
  mode: "enforce",
  onDecision: (event, decision) => {
    const verdict = decision?.block ? `BLOCK (${decision.reason})` : "allow";
    console.log(`  decision for ${event.toolName}: ${verdict}`);
  },
});
const ctx = { hasUI: false, confirm: async () => false };
function status() {
  return manager.formatStatus().replace(/\n/g, " | ");
}
async function call(id: string, toolName: string) {
  console.log(`\n> agent calls ${toolName}`);
  await manager.handleToolCall({ toolCallId: id, toolName, input: {} }, ctx);
  console.log("  state:", status());
}
// 3. Walk a scenario.
await call("c1", "delete");   // blocked: no backup yet
await call("c2", "backup");   // allowed, but deferred (not fired yet)
console.log("  (backup tool is running...)");
manager.handleToolResult({
  toolCallId: "c2", toolName: "backup", input: {}, isError: false,
});
console.log("  backup succeeded -> deferred transition fired");
console.log("  state:", status());
await call("c3", "delete");   // allowed now
await call("c4", "delete");   // blocked again: needs a new backup
await call("c5", "rm");       // always blocked
await call("c6", "push");     // 1 of 2
await call("c7", "push");     // 2 of 2
await call("c8", "push");     // over the limit
await call("c9", "ls");       // no net cares: passes
console.log("\nSystem prompt the LLM would see:");
console.log(manager.formatSystemPrompt());
```

```
EOF
```

Paste the whole block including the final EOF line. Git Bash will create the file.



What the script touches. compile() is from rules; everything else is the gate package.

Step 4.2 Run it and read the verdicts

```
bun run labs/gate-playground.ts
```

Expected output (abridged, with arrows added as commentary)

```
Compiled nets: [ "require-backup-before-delete", "block-rm", "limit-push-2" ]
Verification (reachable states per net): [ {name: "require-backup-before-delete", reachableS
    tates: 3},
  {name: "block-rm", reachableStates: 2}, {name: "limit-push-2", reachableStates: 4} ]
> agent calls delete
  decision for delete: BLOCK (delete requires a successful call to backup first.)
  state: require-backup-before-delete: ready:1 | block-rm: ready:1 | limit-push-2: ready:1,
    budget:2
> agent calls backup
  decision for backup: allow
  state: require-backup-before-delete: ready:1 | ...      <- not moved yet (deferred)
  (backup tool is running...)
  backup succeeded -> deferred transition fired
  state: require-backup-before-delete: gate:1 | ...       <- now moved
> agent calls delete
  decision for delete: allow
  state: require-backup-before-delete: ready:1 | ...      <- back to ready
> agent calls delete
  decision for delete: BLOCK (delete requires a successful call to backup first.)
> agent calls rm
  decision for rm: BLOCK (rm is blocked and cannot be called.)
> agent calls push
  decision for push: allow      state: ... limit-push-2: ready:1, budget:1
> agent calls push
  decision for push: allow      state: ... limit-push-2: ready:1
> agent calls push
  decision for push: BLOCK (push has reached its limit of 2 calls per session.)
> agent calls ls
  decision for ls: allow
System prompt the LLM would see:
## Active Petri Nets (composed)
### require-backup-before-delete
Available gated tools: backup
State: ready:1
...
```

WHAT IS EXECUTING

Line by line: `compile()` parses three lines and returns three SkillNets plus their state counts. `createGateManager` creates one GateState per net and runs autoAdvance, which is why every net already shows 'ready:1' and not 'idle:1'. Each `handleToolCall` runs classifyNets: for 'delete' the first net has jurisdiction but no enabled transition (blocked), the other two abstain. For 'backup' the first net is gated with a deferred transition, so the call is allowed but the marking waits for `handleToolResult`. 'ls' appears in no net, so all three abstain and it passes. The push budget is visible as a token count that drops to zero.

IN SIMPLE WORDS Things to try next

Change *isError: false* to *isError: true* on the handleToolResult line and run again: delete stays blocked because a failed backup does not count. Add a line *require human-approval before push* to the rules: every push is now blocked because the script passes hasUI: false. Change *mode: "enforce"* to *"shadow"*: onDecision still prints BLOCK but nothing is actually blocked.

Part 5. The viewer, the comparison reports, and the agents

Step 5.1 The interactive viewer

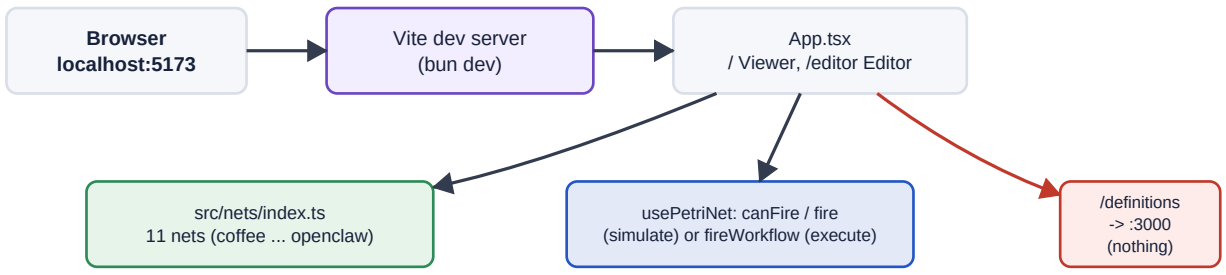
```
bun dev
```

Leave this running. Press Ctrl+C to stop it. Open a second Git Bash window for other commands.

Expected output

```
@petriflow/viewer:dev: $ vite
@petriflow/viewer:dev:   VITE v6.4.1  ready in 3101 ms
@petriflow/viewer:dev:   >  Local:   http://localhost:5173/
```

Open **http://localhost:5173/** in a browser. Pick a net from the selector (coffee is a good start). Click an enabled transition to fire it and watch the tokens move. Switch between simulate mode (pure petri-ts canFire/fire) and execute mode (runs the real execute functions, so coffee takes seconds). The analysis panel shows reachable states live.



Save/load in the editor calls an API on port 3000 that does not exist in this repo. Everything else works offline.

The viewer architecture. It imports the same workflow definitions the tests use.

WHAT IS EXECUTING

`turbo run dev --filter=@petriflow/viewer` starts Vite in apps/viewer. apps/viewer/src/nets/index.ts lists 11 nets, including the four OpenClaw scenarios. The hook usePetriNet.ts holds the marking in React state and calls `canFire`/`fire` or `fireWorkflow`. The /editor page lets you draw a net; its Save button calls /definitions, which Vite proxies to port 3000 where nothing listens, so saving fails quietly. Drawing and analysing still work.

Step 5.2 The OpenClaw safety comparison report

```
bun run comparisons/openclaw-safety/src/index.ts
```

Expected output (abridged)

```
  1. Tool Approval Gate
                          OpenClaw (policy)             PetriFlow (structure)
  Approach                Runtime check: if (!approvalId Structural: execShell requires ...
  Reachable states        N/A                           26
  Terminal states         N/A                           2
  Deadlock-free           N/A                           YES
  2. Message Gating       ...  9 states
  3. Budget & Escalation  ...  21 states
  4. Sandbox Isolation    ...  30 states
  4b. Locked Sandbox: Host places always 0: YES (proven), 21 states
```

WHAT IS EXECUTING

Four workflow definitions in comparisons/openclaw-safety/src/scenarios are analysed with `analyse()`; the locked variant additionally walks every reachable state and checks the host places are zero. No OpenClaw code runs. The table is the author's framing of policy checks versus structure (first guide, chapter 11 discusses the fairness of this).

Step 5.3 Verify the 56-rule ops policy

```
bun run examples/05-openclaw-ops/verify.ts
```

Expected output (abridged)

```
Rules: 56 nets from ops-assistant.rules

Rule verification:
  block-gateway: 2 reachable states
  block-sudo: 2 reachable states
  require-read-before-write: 3 reachable states
  require-git-commit-before-git-push: 3 reachable states
  approve-before-npm-publish: 2 reachable states
  ...
Deployment pipeline: N reachable states
  Token conservation (1 token always): true
Total: 57 nets, NN unique gated tools
```

WHAT IS EXECUTING

`loadRules` compiles a 56-line .rules file. This is the largest rule set in the repo and shows the compiler's scale: 56 independent nets, each 2 or 3 states, verified in milliseconds. Then a hand-written deployment pipeline net is analysed with a token-conservation invariant. Open examples/05-openclaw-ops/ops-assistant.rules to read a realistic policy.

Step 5.4 Browse the documentation site locally (optional)

```
cd site && bun install && bun run dev
# when done: Ctrl+C, then  cd ..
```

WHAT IS EXECUTING

The site is a separate Astro project with its own lockfile. It contains the six tutorials and the package reference pages. It is static; no repo code executes.

Step 5.5 Run a real LLM agent against the gate (optional, needs an API key)

NOTE Before you do this on a company laptop

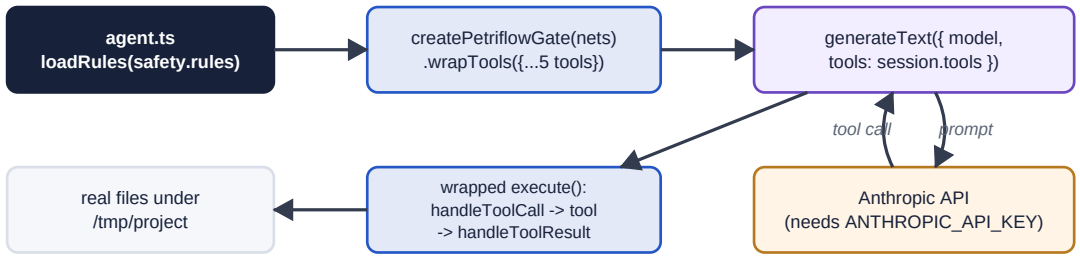
This sends prompts to Anthropic's API and costs a few cents per run. Check your company's policy on external AI services first. The example also creates and deletes files under /tmp/project, so make that folder on your drive as shown.

```
export ANTHROPIC_API_KEY=sk-ant-...your key...
mkdir -p /c/tmp/project && echo hello > /c/tmp/project/temp.log && echo old > \
  /c/tmp/project/old-backup.tar.gz
bun run examples/01-file-management/agent.ts
```

Typical output (the model's wording varies)

```
Loaded rules: [ { name: "require-backup-before-delete", reachableStates: 3 }, { name: "block
    -rm", reachableStates: 2 } ]
BLOCKED delete: delete requires a successful call to backup first.
BLOCKED rm: rm is blocked and cannot be called.

--- Final Response ---
I listed the files, backed up temp.log, deleted it, and I cannot use rm because it is blocke
    d...
```



Up to 10 model steps (stopWhen: stepCountIs(10)). Blocked calls throw ToolCallBlockedError, which the SDK feeds back to the model as text.

The example agent. The gate sits inside the tool's execute function, so the SDK and the model never see an unguarded tool.

WHAT IS EXECUTING

agent.ts loads safety.rules, wraps five tools with `gate.wrapTools`, puts `session.systemPrompt()` into the system prompt, and calls `generateText` with up to 10 steps. When the model calls delete first, the wrapped execute throws ToolCallBlockedError; the SDK turns that into an error result the model reads, and the model then calls backup. onDecision prints each block. The other examples (02 to 04) need more keys (Slack, Discord, email); see examples/.env.example. workflows/simple-agent/run-llm.ts shows an LLM as the engine's decision provider instead.

Part 6. Break a proof on purpose

The fastest way to understand what the tests prove is to make them fail. This experiment adds a shortcut that bypasses the human approval in the agent benchmark, then watches which checks notice.

Step 6.1 Add a bypass transition and run the proofs

```
BYPASS='{ name: "bypassApproval", type: "automatic", inputs: ["codePending"],'
BYPASS="$BYPASS"' outputs: ["codeDone"], guard: null },'
sed -i "/=== Join + Decision ===/i\\    $BYPASS" \
  workflows/agent-benchmark/definition.ts
grep -n bypassApproval workflows/agent-benchmark/definition.ts
bun test workflows/agent-benchmark 2>&1 | grep -E '\(fail\)|pass$|fail$'
```

Expected output

```
207:    { name: "bypassApproval", type: "automatic", inputs: ["codePending"], outputs: ["cod
    eDone"], guard: null },
(fail) agent-benchmark: formal properties > code execution always passes through humanApprov
    al [1.01ms]
 8 pass
 1 fail
```



The test 'code execution always passes through humanApproval' looks for exactly this shortcut and fails. The CLI --strict still passes: it only checks deadlocks and invariants, not domain properties. That difference is the lesson.

The shortcut you added, and why one test fails while the CLI stays green.

WHAT IS EXECUTING

The failing test loops over every transition whose inputs include codePending and asserts its outputs do not include codeDone. Your new transition violates that. The other eight properties still hold: the net still terminates, the join still needs three tokens, the budget is still bounded. Reachable states stay 196 because the new path reaches markings that other paths already reached.

```
bun packages/cli/src/cli.ts analyse workflows/agent-benchmark/index.ts \
  --strict; echo "exit code: $?"
```

Expected output

```
Workflow: agent-benchmark

  Reachable states: 196
  Terminal states:  4 (all valid)
    v responseGenerated, iterationBudget(3)
    ...
exit code: 0
```

LOOPHOLE The CLI did not notice

--strict only checks for unexpected terminal states and invariant violations. A bypassed human gate is neither. Domain properties like 'code never runs without approval' live only in the test file. If you adopt this engine, the property tests are the real safety net, and they must be written per workflow.

Step 6.2 Put it back

```
git checkout -- workflows/agent-benchmark/definition.ts
bun test workflows/agent-benchmark 2>&1 | grep -E 'pass$|fail$'
```

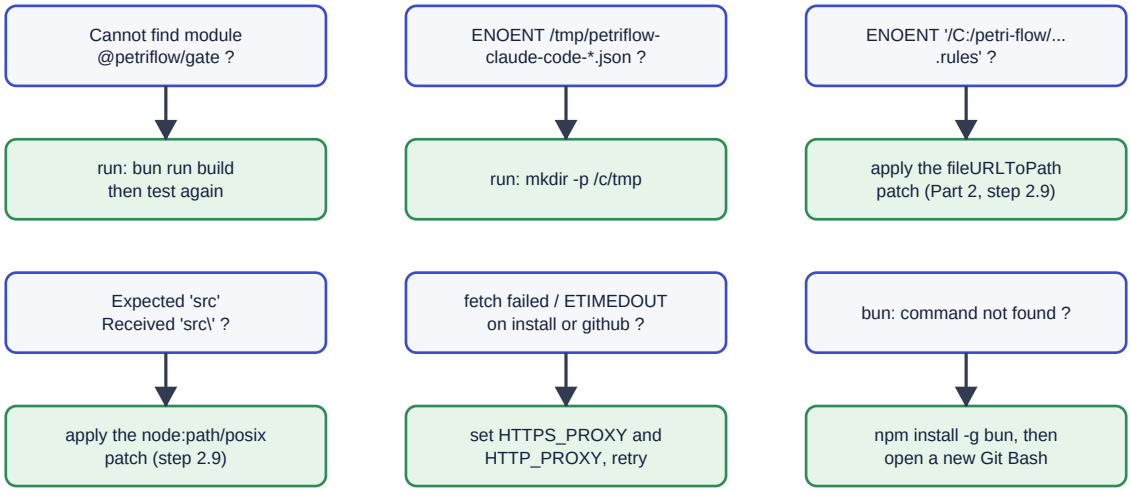
Expected output

```
 9 pass
 0 fail
```

Other experiments in the same spirit: remove *deferred: true* from compileSequence in packages/rules/src/compiler.ts and watch the 'require A before B' tests fail when the backup result is an error. Change ITERATION_BUDGET in workflows/agent-benchmark/definition.ts to 5 and watch the reachable state count grow (and the benchmark's budget test adapt, because it reads the constant).

Part 7. Troubleshooting, cleanup, cheat sheet

7.1 If something goes wrong



The six problems you are most likely to meet, and the fix for each.

Symptom	Cause	Fix
error: Cannot find module '@petriflow/gate'	Packages not built; imports resolve to dist/	bun run build
ENOENT ... /tmp/petriflow-claude-code-*.json	C:\tmp does not exist	mkdir -p /c/tmp
ENOENT ... '/C:/petri-flow/.../safety.rules'	URL pathname on Windows	patch 2 in step 2.9
Expected: "src" Received: "src\"	path.normalize uses backslashes	patch 1 in step 2.9
fetch failed / ETIMEDOUT / ECONNRESET	Proxy or firewall	export HTTPS_PROXY=... then retry; github-lookup live test may still be blocked
Port 5173 is in use	Another Vite is running	close it, or run: cd apps/viewer && bunx vite --port 5174
bun: command not found after install	PATH not refreshed	open a new Git Bash window
warning: LF will be replaced by CRLF	Git autocrlf on Windows	harmless; or git config core.autocrlf input
Tests very slow the first time	Windows Defender scanning node_modules	add the repo folder to Defender exclusions if IT allows
script "dev" exited with code 143	You pressed Ctrl+C	normal shutdown message

7.2 Leaving the repo clean

```
git status --short          # shows your patches and labs/
git checkout -- .           # undo the step 2.9 patches and step 6.1 edit
rm -rf labs                 # remove the playground if you do not want it
git status --short          # should print nothing
```

node_modules, dist and .turbo are listed in .gitignore, so they never show up in git status. Delete them with `rm -rf node_modules packages/*/dist apps/viewer/dist .turbo` if you need the disk space back.

7.3 One-page cheat sheet, in order

```
# Part 1: setup
git clone https://github.com/joshuaisaact/petri-flow.git && cd petri-flow
bun install
bun run build
bun run check
mkdir -p /c/tmp

# Part 2: tests
bun test packages/engine
bun test packages/gate
bun test packages/rules
bun test packages/vercel-ai
bun test packages/cli
bun test packages/agent-sdk packages/pi-extension packages/openclaw packages/pi-assistant
bun test packages/claude-code
bun test workflows
bun test comparisons
sed -i 's|from "node:path";|from "node:path/posix";|' packages/pi-assistant/src/nets/cleanup.ts \
  packages/pi-extension/src/nets/nuke.ts
FIX='s|new URL(\(.*\), import.meta.url).pathname|Bun.fileURLToPath(new URL(\1, import.meta.url))|'
sed -i "$FIX" examples/0*/agent.test.ts
bun test examples
bun test                      # 639 pass

# Part 3: run things
bun run workflows/coffee/index.ts
bun run workflows/order-checkout/index.ts
bun run workflows/simple-agent/index.ts
bun run workflows/agent-benchmark/index.ts
bun run workflows/github-lookup/index.ts
bun packages/cli/src/cli.ts analyse workflows/coffee/index.ts
bun packages/cli/src/cli.ts analyse workflows/order-checkout/index.ts --strict
bun packages/cli/src/cli.ts analyse workflows/agent-benchmark/index.ts --json | head
bun packages/cli/src/cli.ts analyse workflows/coffee/index.ts --dot

# Part 4: gate playground (create labs/gate-playground.ts first, see 4.1)
bun run labs/gate-playground.ts

# Part 5: viewer and reports
bun dev                       # http://localhost:5173
bun run comparisons/openclaw-safety/src/index.ts
bun run examples/05-openclaw-ops/verify.ts

# Part 6: break and restore
bun test workflows/agent-benchmark
git checkout -- .
```

7.4 Where each command's code lives

You ran	Entry file	Key functions in order
bun run build	turbo.json, packages/*/tsup.config.ts	turbo graph -> tsup build per package
bun test	/src/*.test.ts	bun:test describe/it -> the functions named in Part 2
bun run workflows/X/index.ts	workflows/X/index.ts	defineWorkflow -> createExecutor -> new Scheduler -> createInstance -> tick (step -> enabledWorkflowTransitions -> fireWorkflow -> execute) -> inspect

You ran	Entry file	Key functions in order
cli.ts analyse	packages/cli/src/cli.ts	import(workflow) -> analyse -> toNet -> petri-ts analyse -> formatAnalysis / formatJson
labs/gate-playground.ts	labs/gate-playground.ts	compile -> createGateManager -> composedToolCall (classifyNets, phases) -> handleToolResult -> autoAdvance
bun dev	apps/viewer/src/main.tsx	App -> Viewer -> usePetriNet -> canFire/fire or fireWorkflow
examples/01 agent.ts	examples/01-file-management/agent.ts	loadRules -> createPetriflowGate -> wrapTools -> generateText -> wrapped execute -> handleToolCall/Result
openclaw-safety index.ts	comparisons/openclaw-safety/src/index.ts	analyse x4 -> reachableStates on the locked variant -> print table

End of lab guide.